

Prise de note

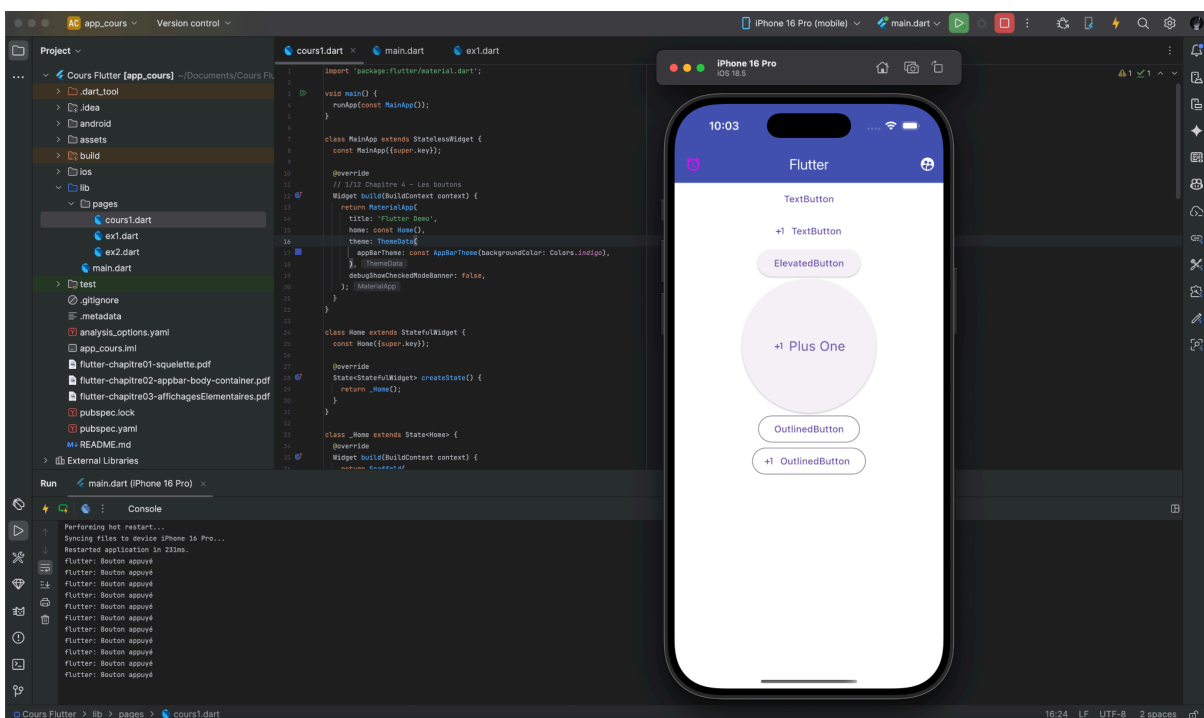
Chapitre 4 : Les boutons — Flutter

Prise de note du **22/10/25**

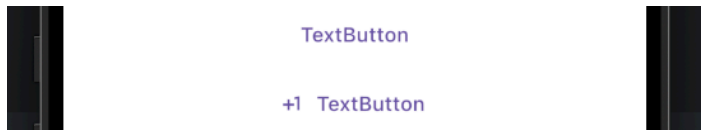


Au cours de ce chapitre, nous avons travaillé sur les différents types de boutons dans Flutter et sur la gestion des états. L'objectif était de comprendre comment créer des boutons, leur attribuer des actions et rendre l'interface dynamique.

On a commencé par créer un projet Flutter de base. Dans ce projet, on a ajouté plusieurs types de boutons : **TextButton**, **ElevatedButton**, **OutlinedButton**, **IconButton** et **FloatingActionButton**.



Le **TextButton** est un bouton simple, composé uniquement de texte.



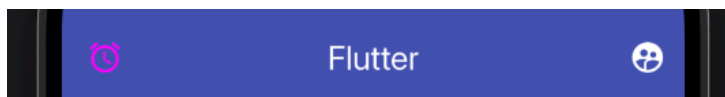
Le **ElevatedButton** a un relief, il sert généralement pour les actions principales.



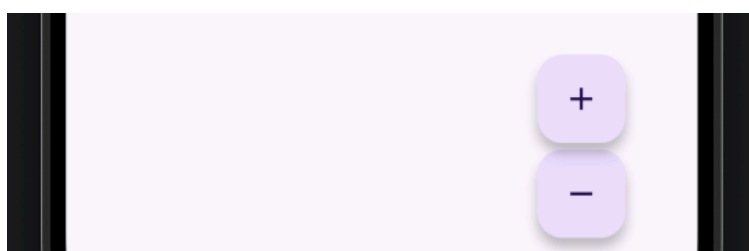
Le **OutlinedButton**, lui, est plus discret. Il a une bordure fine et se prête bien aux actions secondaires.



L'**IconButton** ne contient qu'une icône et s'utilise souvent dans les barres d'outils, par exemple pour les paramètres, la recherche ou le profil utilisateur.



Enfin, le **FloatingActionButton** est le bouton flottant en bas à droite de l'écran. Il représente l'action principale de la page, par exemple "ajouter un élément".



Tous ces boutons appelaient la même fonction **btDown()** pour afficher un message dans la console “Bouton appuyé” pour vérifier leur fonctionnement.

```
void btDown() {  
  setState(() {  
    print('Bouton appuyé');  
  });  
}
```

Nous avons aussi utilisé un **ConstrainedBox** pour fixer les dimensions d’un bouton. Cela permet de contrôler précisément sa taille, son apparence et sa position.

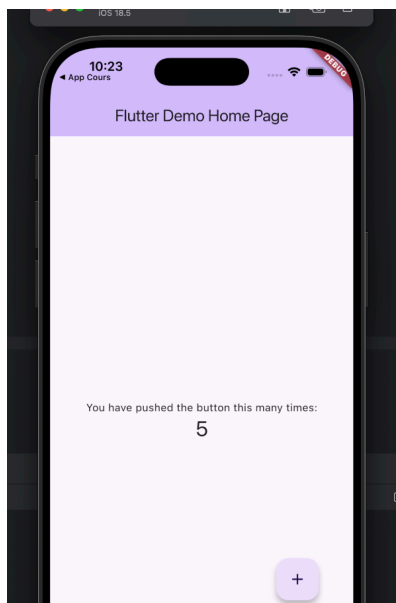
```
ConstrainedBox(  
  constraints:  
    const BoxConstraints.tightFor(width: 200, height: 200),  
  child: ElevatedButton.icon(  
    onPressed: () {},  
    label: const Text('Plus One'),  
    icon: const Icon(Icons.plus_one),  
    style: ElevatedButton.styleFrom(  
      padding: const EdgeInsets.symmetric(  
        horizontal: 30, vertical: 10),   
        EdgeInsets.symmetric  
      ),  
      textStyle: const TextStyle(fontSize: 20),  
      shape: const CircleBorder(),  
    ),  
  ),  
), ElevatedButton.icon
```

Dans la deuxième partie du chapitre, nous avons abordé la gestion des **états** dans Flutter à l'aide de l'exemple du projet de base (celui avec le compteur). Cet exemple nous a servi à comprendre comment fonctionne la méthode `setState()`.

Dans le code, on a une variable `_counter` initialisée à 0 et une fonction `_incrementCounter()` qui l'incrmente à **chaque clic**. Quand on appelle `setState()`, Flutter **reconstruit automatiquement l'interface pour afficher la nouvelle valeur**.

```
class _Ex1State extends State<Ex1> {  
  int _counter = 0;  
  
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }  
}
```

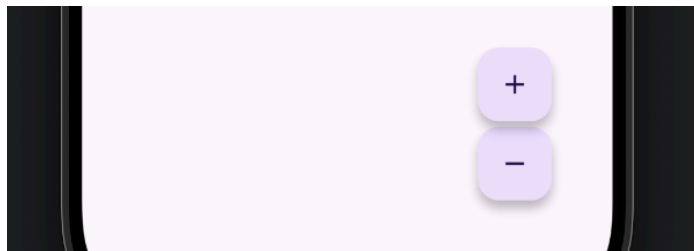
On a pu constater que la ligne `Text('$ _counter')` affiche le nombre de clics directement à l'écran.



On a aussi étudié la notion de thème global dans Flutter. Grâce à `Theme.of(context)`, on peut récupérer les couleurs et styles définis dans `MaterialApp`. Par exemple, `colorScheme.inversePrimary` sert à choisir une couleur contrastante pour le fond de l'`AppBar` ou du `FloatingActionButton`.

Ensuite, on a vu comment placer plusieurs `FloatingActionButton` sur une même page.
Par défaut, le `Scaffold` n'accepte qu'un **seul bouton flottant**.

Pour en avoir plusieurs, on a utilisé un **Stack**, qui permet de superposer des widgets, et des `Positioned` pour les placer à des endroits précis de l'écran.

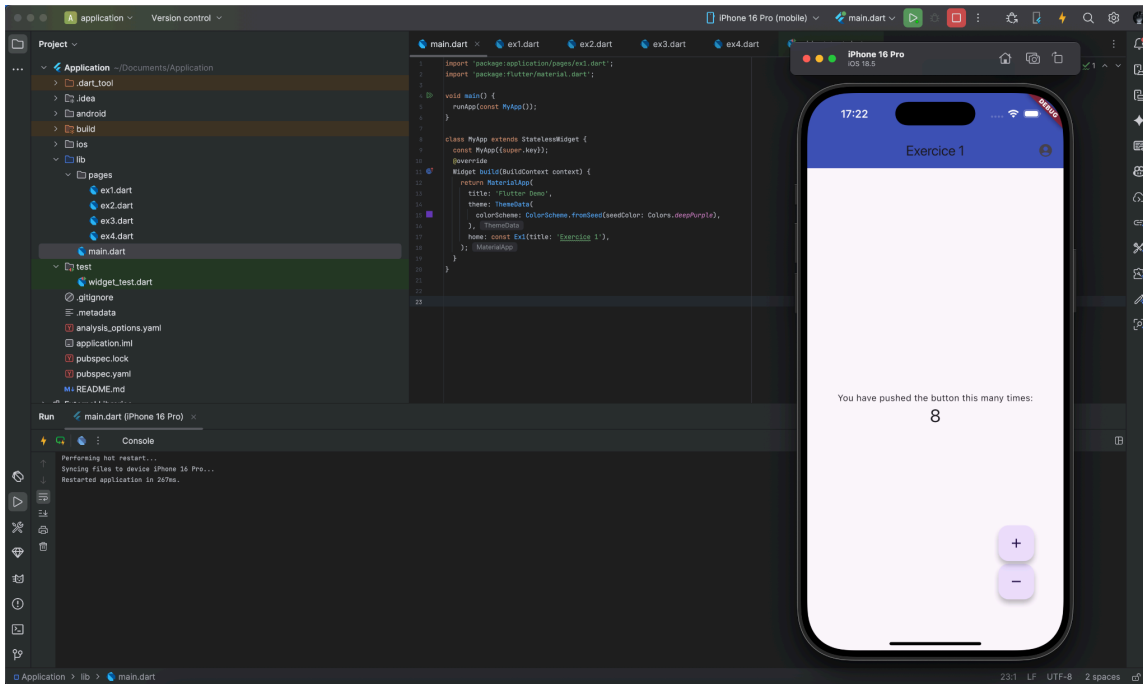


Grâce à cette approche, on a pu créer une interface avec plusieurs actions flottantes : un bouton “+” pour ajouter et un bouton “-” pour retirer, par exemple.

Exercices :

En fin de chapitre, on a fait quelques exercices pour mettre tout ça en pratique :

- Créer un bouton flottant supplémentaire à l'exemple de base avec une icône décrémentation (ex1)



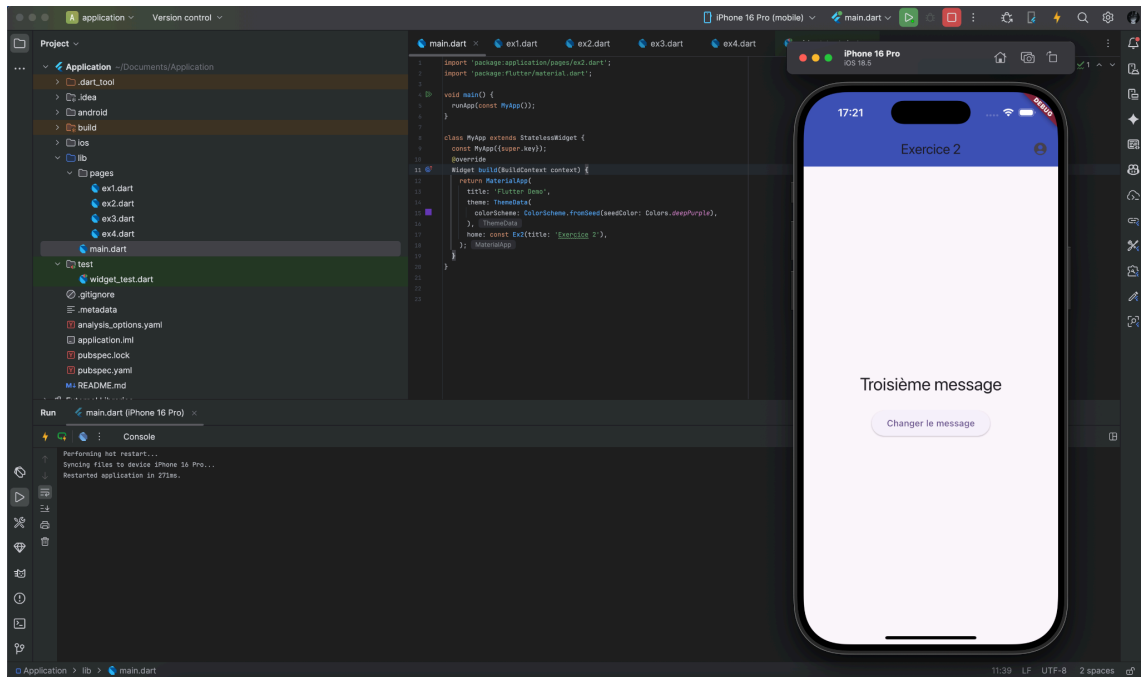
Dans cet exercice, nous avons mis en pratique l'utilisation des boutons en ajoutant un nouveau bouton flottant, ainsi qu'une logique permettant de décrémenter le compteur.

Pour ce faire nous avons dû mettre en place un nouveau bouton **FloatingActionButton** pour décrémenter le compteur. Et nous avons dû ajouter une nouvelle logique pour décrémenter le compteur, en implémentant cette fois une méthode **decrement()** dédiée à cette action.

```
class _Ex1State extends State<Ex1> {  
  int _counter = 0;  
  
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }  
  
  void _decrementCounter() {  
    setState(() {  
      if (_counter > 0) {  
        _counter--;  
      }  
    });  
  }  
}
```

```
floatingActionButton: Stack(  
  fit: StackFit.expand,  
  children: [  
    Positioned(  
      right: 30,  
      bottom: 30,  
      child: FloatingActionButton(  
        tooltip: 'Increment',  
        onPressed: _decrementCounter,  
        child: const Icon(Icons.remove),  
        heroTag: null,  
      ),  
    ),  
    Positioned(  
      right: 30,  
      bottom: 90,  
      child: FloatingActionButton(  
        tooltip: 'Decrement',  
        onPressed: _incrementCounter,  
        child: const Icon(Icons.add),  
        heroTag: null,  
      ),  
    ),  
  ],  
)
```

- Créer un bouton qui change le texte a chaque clique dans une liste (ex2)



Dans cet exercice, nous avons créé une liste de messages. À chaque appui sur le bouton, nous faisons varier l'index pour l'incrémenter et ainsi afficher le message correspondant dans la liste. Lorsque l'index atteignait la fin de la liste, il revenait au premier message (index 0).

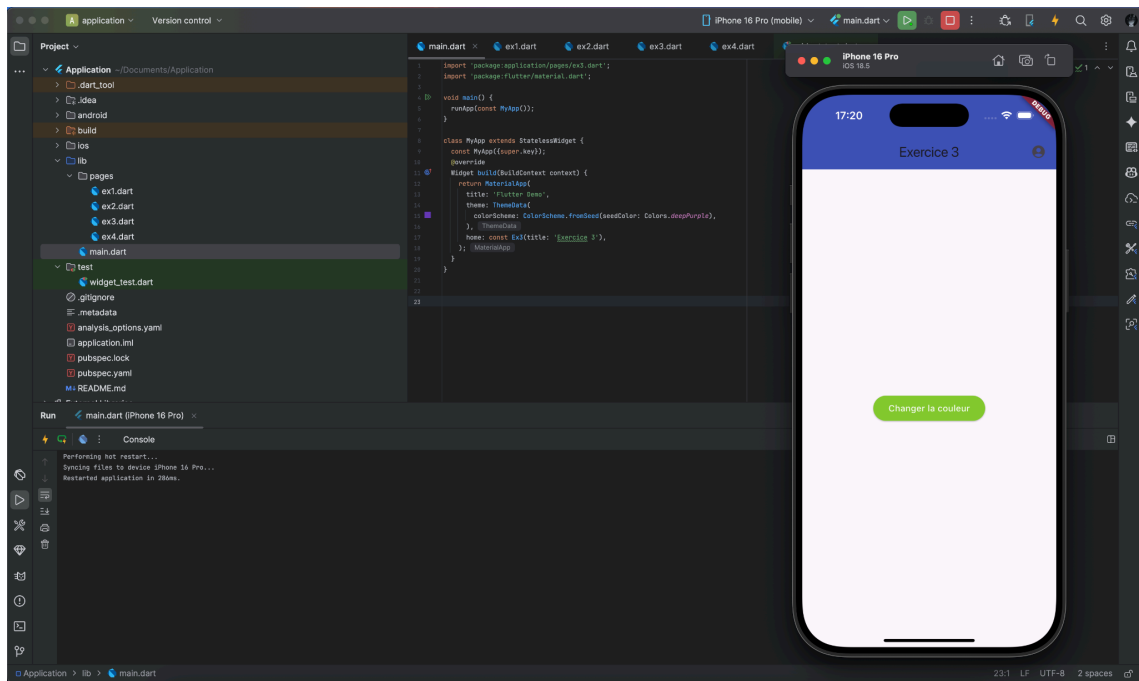
Pour cela, nous avons mis en place une liste de mots et utilisé un index afin de changer le message affiché à chaque clic.

```
class _Ex2State extends State<Ex2> {
  final List<String> messages = [
    "Premier message",
    "Deuxième message",
    "Troisième message",
    "Dernier message"
  ];
  int index = 0;

  void _changeMessage() {
    setState(() {
      index = (index + 1) % messages.length;
    });
  }
}

body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text(
        messages[index],
        style: Theme.of(context).textTheme.headlineMedium,
        textAlign: TextAlign.center,
      ),
      Text(
        const SizedBox(height: 20),
        ElevatedButton(
          onPressed: _changeMessage,
          child: const Text('Changer le message'),
        ),
      ),
    ],
  ),
),
),
Center
```

- Créer un ElevatedButton pour qu'il change de couleur à chaque clic. (ex3)



Dans cet exercice, lorsque l'on cliquait sur le bouton, il fallait changer sa couleur à chaque appui.

Pour cela, j'ai utilisé **Random()** afin de générer une couleur aléatoire en utilisant un code RGB à chaque clic. Ainsi, le bouton change de couleur à chaque fois qu'on appuit dessus.

```

class _Ex3State extends State<Ex3> {
  Color _buttonColor = Colors.blue;
  final Random _random = Random();

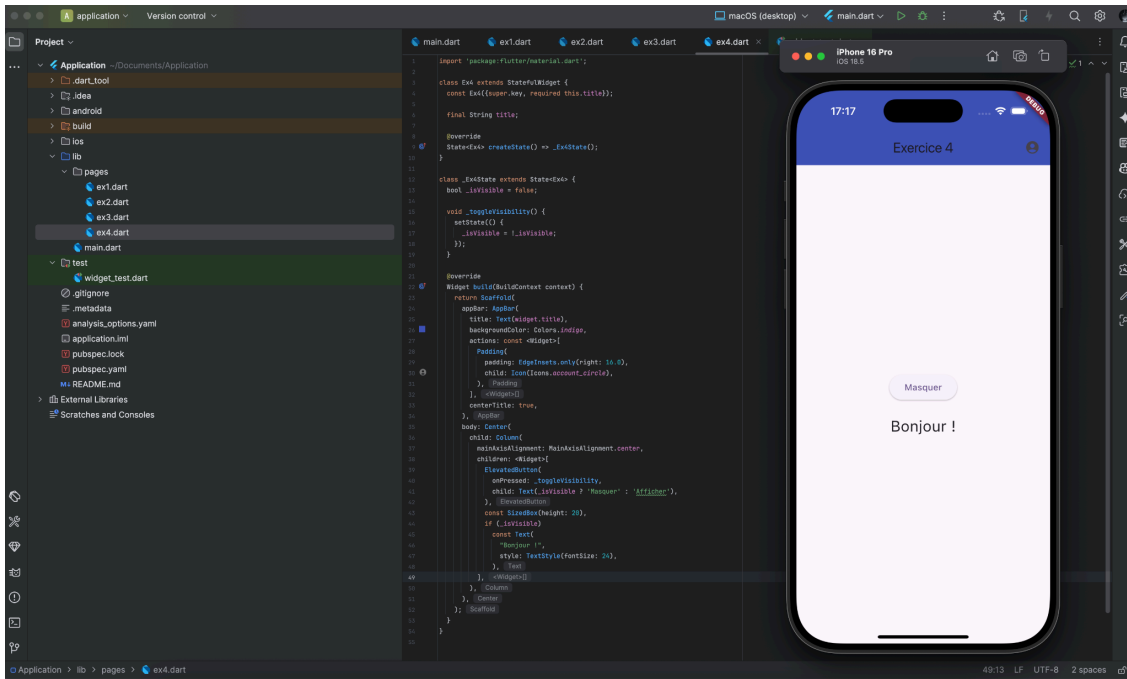
  void _changeColor() {
    setState() {
      _buttonColor = Color.fromRGB0(
        _random.nextInt(256),
        _random.nextInt(256),
        _random.nextInt(256),
        1);
    });
  }
}

body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      ElevatedButton(
        onPressed: _changeColor,
        style: ElevatedButton.styleFrom(
          backgroundColor: _buttonColor,
          foregroundColor: Colors.white),
        child: const Text('Changer la couleur'),
      ),
    ],
  ),
),
), Center

```

(On note que, comme cela n'était pas précisé dans l'exercice, nous pouvions utiliser une couleur générée aléatoirement. Cependant, il aurait également été possible d'utiliser une liste prédéfinie de couleurs et de changer la couleur à chaque clic en utilisant un index pour parcourir cette liste.)

- Créer un bouton qui cache ou affiche le bouton à chaque clique (ex4)



Dans cet exercice, lorsqu'on appuyait sur le bouton, il fallait simplement changer l'état d'affichage du texte **"Bonjour !"**. Quand on cliquait sur **Masquer**, le texte disparaissait, et à l'inverse, quand on appuyait sur **Afficher**, il réapparaissait. Le bouton alternait donc automatiquement entre les deux états, en changeant aussi son texte.

Pour ce faire, il a simplement fallu faire varier l'état d'affichage du texte à chaque clic, ainsi que le texte du bouton. Le bouton alternent donc entre **Afficher** et **Masquer**, affichant ou cachant le texte en conséquence.

```
class _Ex4State extends State<Ex4> {
  bool _isVisible = false;

  void _toggleVisibility() {
    setState(() {
      _isVisible = !_isVisible;
    });
  }
}
```

```
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      ElevatedButton(
        onPressed: _toggleVisibility,
        child: Text(_isVisible ? 'Masquer' : 'Afficher'),
      ), ElevatedButton
    ], ElevatedButton
  ), ElevatedButton
), Column
), Center
```

Ces exercices nous ont permis de manipuler les états, de mieux comprendre comment Flutter reconstruit ses widgets, et d'expérimenter la personnalisation visuelle des boutons.

En résumé, dans ce chapitre, nous avons :

- appris à utiliser et différencier les principaux types de boutons Flutter,
- compris comment gérer les états avec `setState()`,
- et vu comment placer plusieurs boutons sur un écran.